DFSha — Fase II

Revisión técnica del prototipo grupal · opción 1: cliente/servidor S2S

Andrés Felipe Martínez Velasquez · Juan José García Duque · Samuel Parra Cadavid

Universidad Pontificia Bolivariana · Profesor Álvaro Ospina Sanjuan

Estado: desplegado y validado en AWS Academy (cuatro VMs EC2). Evidencias en dfsha/resultados.

1. Alcance y estado verificable

Esta versión corrige el prototipo tras contrastarlo con la consigna y los tres diseños individuales. Se desplegó en cuatro VMs EC2 de AWS Academy con deploy/desplegar.py y allí se ejecutaron la suite de regresión, las caídas de servicios y el benchmark. Las capturas son de la terminal del equipo conectada por SSH al bastión. Las mediciones de la versión anterior se conservan como históricas.

2. Comparación y elección arquitectónica

Criterio	Andrés	García	Samuel	Revisión
Control	2 APIs + etcd/Raft	PostgreSQL activo/pasivo	3 coordinadores + etcd	2 APIs + 3 etcd
Bloque	8 MiB	4 MiB	4 MiB	8 MiB; comparación pendiente
Réplicas	R3/W2	R2/W2	R3/W2	R3/W2, recibos verificados
Concurrencia	Por bloque	Bloque + versión esperada	Por archivo + CAS	rw por bloque; w/a exclusivos
Lectores	Snapshot; gracia	Manifiesto estable	Lectores renovables	Lease + retención por referencia
Seguridad	Argon2id/RS256/ mTLS	Argon2id/RS256/ mTLS	Argon2id/TLS/m TLS	scrypt/EdDSA/id entidades separadas

La topología de Andrés es una base defendible por disponibilidad del catálogo y paralelismo. Se incorporan la validación de base de García y los leases lectores de Samuel. S2S y snapshots son decisiones coincidentes: no se atribuyen exclusivamente a una persona. El detalle con las secciones originales de cada propuesta está en COMPARACION_PROPUESTAS.md.

3. Decisiones y límites del contrato

Se mantiene REST/HTTPS para control, gRPC/mTLS para datos, tres etcd [3], dos APIs equivalentes y cuatro nodos, con separación de planos inspirada en GFS/HDFS [1], [2]. Los bloques miden hasta 8 MiB; el objetivo es R=3 y la preparación exige dos recibos de nodos distintos. La réplica S2S se hace desde un bloque validado, secuencialmente: no se presenta como pipeline solapado. ADR_REVISION.md documenta los cambios y sus costos.

La publicación usa un epoch global y CAS de sesión/referencias. Esto prioriza corrección, pero introduce contención de metadatos y no promete escalabilidad ilimitada. MAX_BLOCKS=8192;

lock y rename atómicos hasta 48 bloques/entradas. El catálogo v2 no migra automáticamente el formato previo.

4. Componentes y despliegue

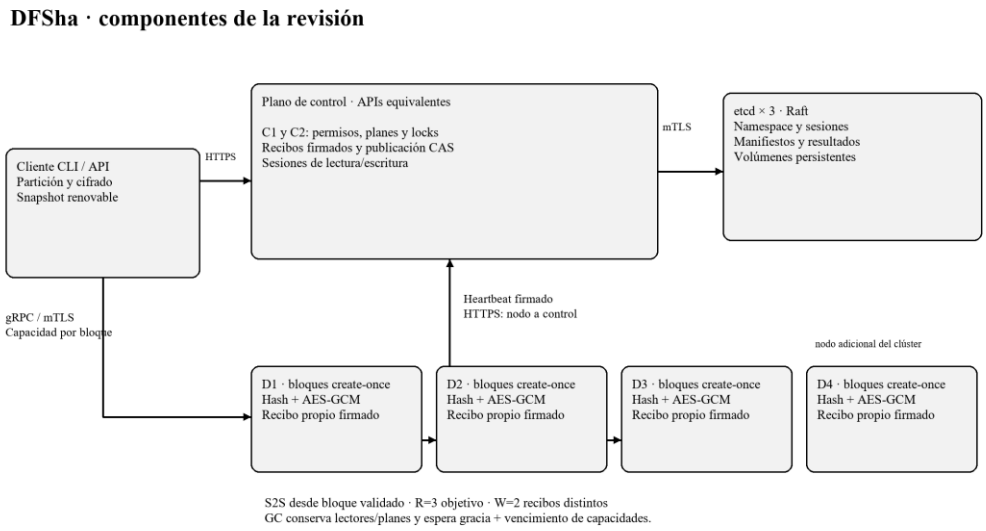


Figura 1. Componentes corregidos: nodos sin acceso al catálogo, recibos firmados y lectores con lease.

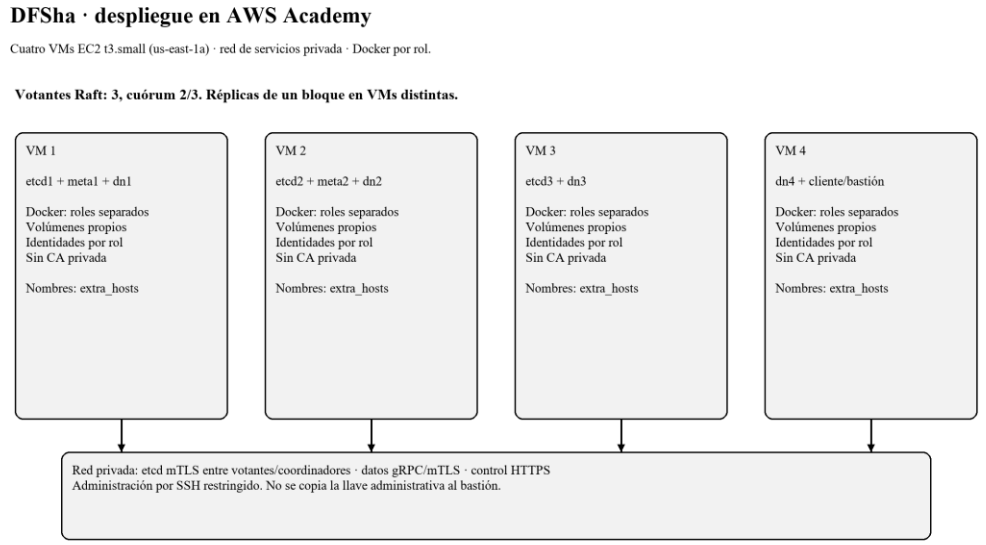


Figura 2. Despliegue en AWS Academy: tres etcd, dos APIs y cuatro nodos de datos en cuatro VMs.

5. Protocolos, datos y algoritmos

Estado/operación	Contrato revisado
open	Valida propietario; fija snapshot; crea sesión con lease de 60 s renovable.
Reserva	Asigna ID nuevo y destinos; registra plan por bloque.

Estado/operación	Contrato revisado
authorize	Fija SHA-256 y tamaño cifrado; emite capacidad ligada a operación y nodos.
WriteBlock	Consume stream incremental, valida, fsync, create-once y firma recibo por nodo.
prepared	Verifica firmas, nodos distintos y contenido del plan; persiste ficha preparada.
close	Valida planes/locks, tamaños y CAS; publica versión y resultado idempotente en txn acotada.
read	Lee snapshot; refresca ubicaciones/capacidades sin cambiar versión; valida hashes y GCM.
GC	Retiene referencias actuales, lectores y planes; marca, espera y valida antes del borrado.

RF1 se cubre con login y el árbol por usuario; RF2 con put/get de archivo completo; RF3 con open/close/read/write/lock y la semántica declarada: create-once por bloque, rw con lock, w/a exclusivos y append sin pasar EOF.

API: /v1/auth/login; /v1/fs/ls,mkdir,rmdir,rm,rename; /v1/files/open; /v1/files/{fd}/keepalive,manifest,lock,unlock,blocks,close,abort,status. Las rutas /blocks/{bid}/authorize y /prepared separan autorización y confirmación. El contrato gRPC [4] está en proto/dfsha.proto.

Ejemplo: POST /v1/files/{fd}/blocks con {"index":0,"request_id":"op-b0"}; authorize fija hash y tamaño; WriteBlock devuelve recibos firmados; prepared recibe {"receipts":["firma-dn1","firma-dn2"]}. Un recibo duplicado/falso recibe 409 BAD_RECEIPT. close con el mismo fd/cuerpo devuelve la versión publicada, sin publicar otra.

6. Consistencia, concurrencia y fallos

rw comprueba el ID base de cada bloque al adquirir el lock. Si otro escritor cambió ese bloque desde open, responde 409 y exige reabrir/releer; bloques distintos pueden avanzar en paralelo. w y a son exclusivos respecto a otros escritores. rm/rename rechazan escritores activos con 423. La expiración de la sesión elimina sus locks; el commit compara la revisión de sesión y no puede publicar con una sesión vencida.

Un lector abierto conserva snapshot con lease renovable [11]; la visibilidad entre aperturas sigue el modelo close-to-open [5]. Al cerrar o vencer, sus referencias dejan de proteger bloques obsoletos. La gracia no sustituye el seguimiento de lectores. El GC no elimina en el ciclo de marcado y espera el horizonte de capacidades para no permitir un replay tras borrar el ID.

Heartbeats firmados cada 3 s: sospechoso a 10 s y caído a 30 s. El monitor renueva liderazgo en paralelo, verifica que siga vigente y replica usando el hash autorizado del catálogo. Una fuente corrupta no se registra como reparada. Raft [9] mantiene el catálogo; con pérdida de cuórum, las operaciones fallan cerrado, priorizando consistencia [10]. No se promete disponibilidad ante cualquier combinación de fallos.

7. Seguridad e integridad

Cada servicio tiene certificado propio. La CA de etcd solo habilita miembros y coordinadores; nodos y cliente no reciben sus credenciales ni KEK. gRPC exige mTLS y capacidad. JWT y capacidades tienen firmantes separados, y cada nodo firma sus recibos. La KEK se carga solo en metadatos. El hash de contraseña usa scrypt con sal (N=32768,r=8,p=3). Estos cambios deben revisarse con el equipo y el profesor cuando corresponda.

AES-256-GCM [8] usa AAD=block_id; JWT [6] transporta identidad y capacidades, sobre canales TLS [7]. Cada bloque se valida por tamaño/hash; todas las versiones tienen hash canónico de manifiesto. put/w añade el SHA-256 plano global; rw/a no lo recalculan y lo declaran null. El hash de manifiesto no se presenta como hash de todo el archivo plano. Descargas usan temporal y solo reemplazan destino tras validar.

8. Pruebas en AWS Academy

Las cuatro VMs EC2 t3.small corren Ubuntu 24.04 y Docker. deploy/desplegar.py entrega a cada VM solo los certificados de sus roles, construye la imagen y levanta los servicios con la red del host. La figura siguiente muestra los contenedores de cada VM, el clúster etcd con su líder y los cuatro nodos vivos.

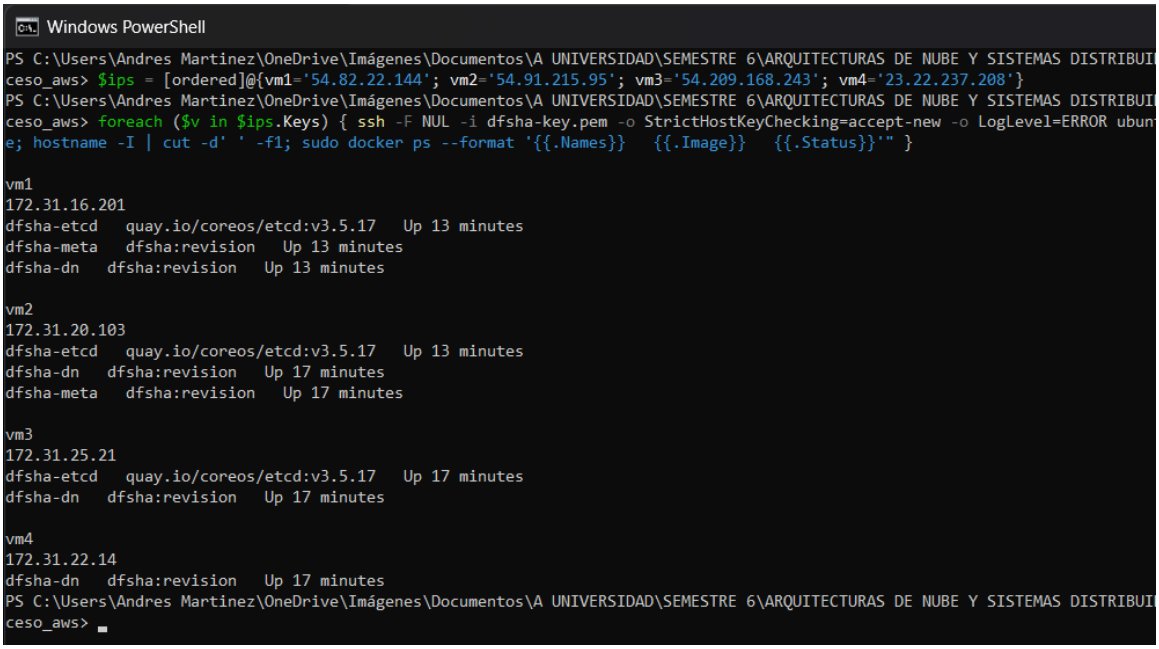


Figura 3. Contenedores por VM, estado de etcd y de los nodos de datos en AWS.

La suite de regresión se ejecutó en el bastión contra el clúster real (revision-20261106-023859): 19 casos, 0 fallos, 0 errores y 0 omitidos. Cubre recibos falsos y duplicados, contenido inmutable, snapshot sin caché, base vieja, bloques distintos en paralelo, truncado vacío, rm/rename con escritores activos, idempotencia del cierre, rangos inválidos, aislamiento por usuario, mTLS sin certificado, clientes concurrentes y la transferencia de 129 bloques (1032 MiB) con hash coincidente.

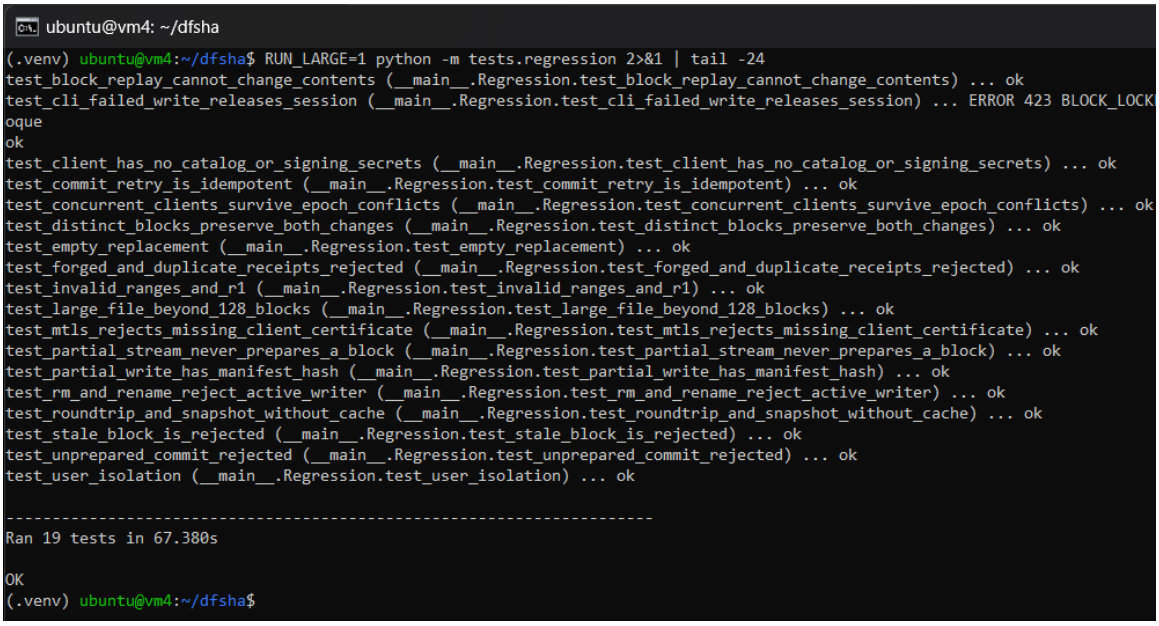


Figura 4. Suite de regresión ejecutada en el bastión de AWS.

RF1, RF2 y RF3 desde la CLI: carga y descarga de un archivo grande con el mismo SHA-256, bloques repartidos entre los cuatro nodos, y write/append/cat con la semántica declarada.

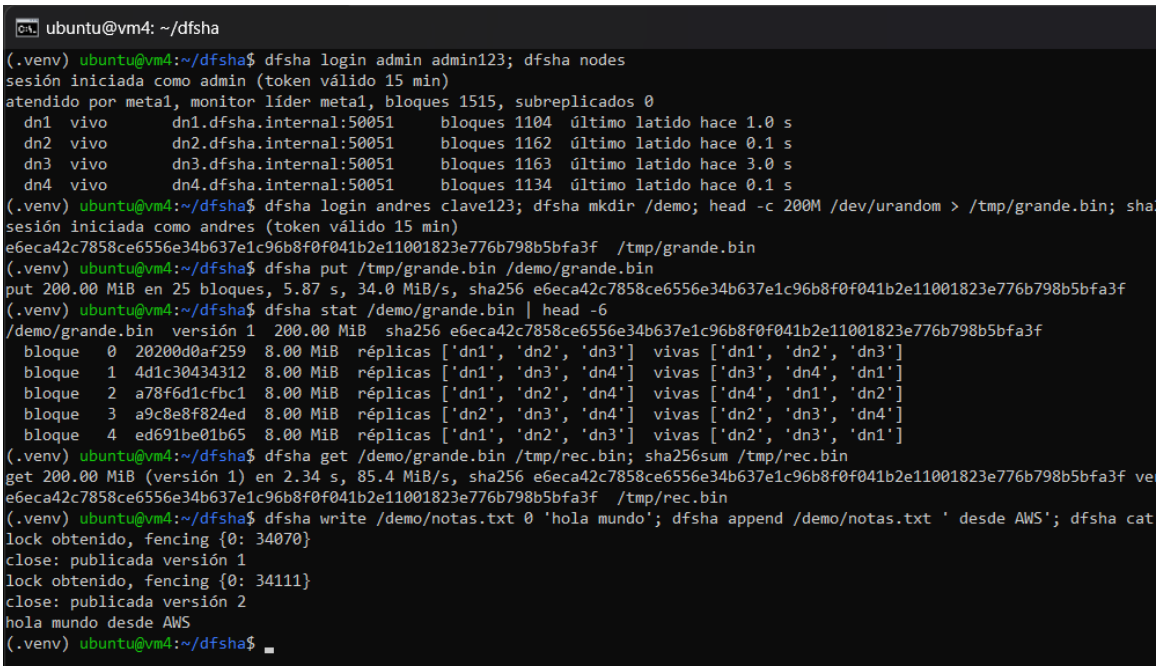


Figura 5. RF1-RF3 en AWS: put, stat, get, write, append y cat.

Concurrencia: el segundo escritor del mismo bloque recibe 423. Si espera el lock (--esperar), recibe 409 VERSION_CONFLICT porque el bloque cambió desde su open y debe reabrir: nunca sobrescribe sobre una base vieja. Dos escritores en bloques distintos del mismo archivo publican los dos cambios.

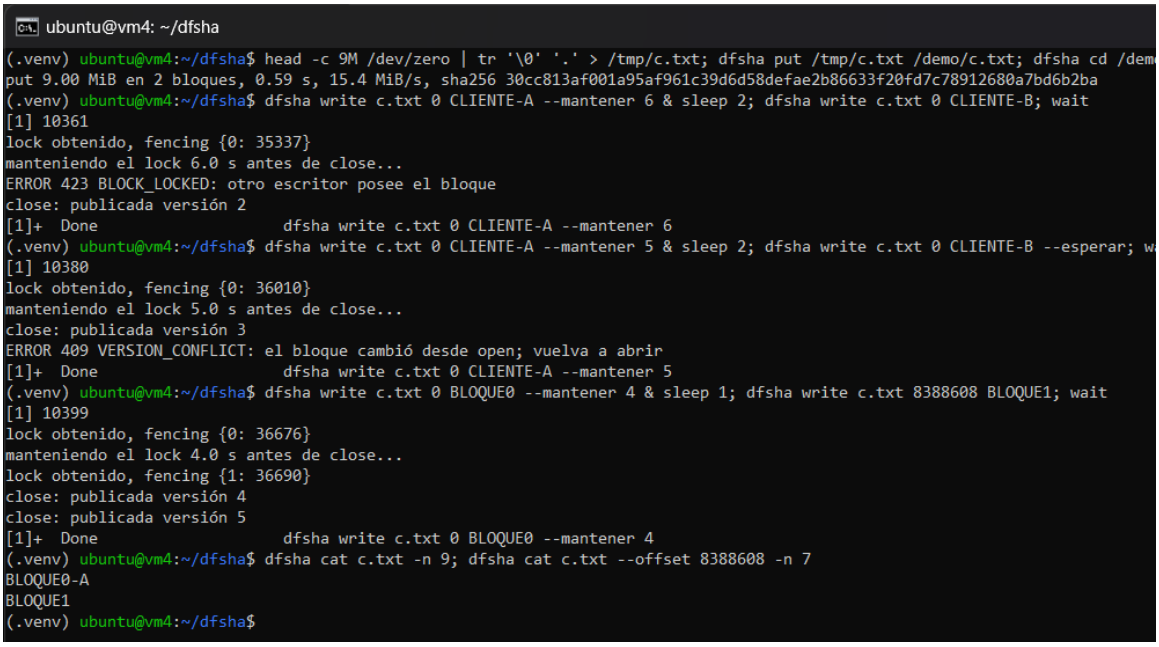


Figura 6. Dos clientes sobre el mismo bloque y sobre bloques distintos.

scripts/verify_aws.py detuvo contenedores reales por SSH y comprobó cada efecto con sondas desde el bastión (revision-failures-aws-abe1be38f48a, 19 pasos registrados, todo restaurado al final):

Paso	Resultado
vm4: sonda seed	OK: seed
vm1: detener dn	dfsha-dn
vm4: sonda check	OK: check
vm4: sonda write	OK: write
vm4: sonda repaired	OK: repaired
vm1: iniciar dn	dfsha-dn
vm1: detener meta	dfsha-meta
vm4: sonda check	OK: check
vm1: iniciar meta	dfsha-meta
vm1: detener etcd	dfsha-etcd
vm4: sonda check	OK: check
vm2: detener etcd	dfsha-etcd
vm4: sonda unavailable	OK: pérdida de cuórum rechazada con 503
vm2: iniciar etcd	dfsha-etcd
vm1: iniciar etcd	dfsha-etcd
vm4: sonda check	OK: check

Con dn1 caído el archivo se leyó y se escribió sin errores, y el monitor rehízo tres copias sin dn1 (la sonda esperó 7 s de reparación). Con meta1 caído el cliente usó meta2. Con un miembro de etcd menos el catálogo siguió operando; con dos, las operaciones fallaron cerrado con 503, priorizando consistencia [10].

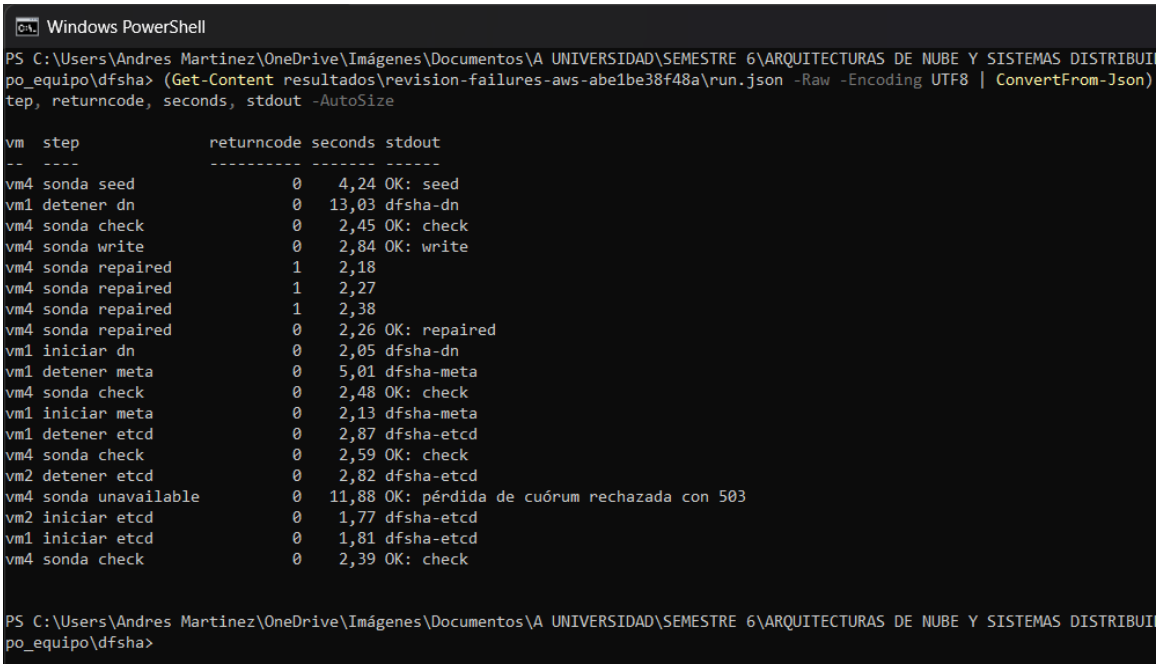


Figura 7. Registro de las caídas ejecutadas en AWS.

Seguridad: sin sesión o con clave incorrecta la respuesta es 401; otro usuario recibe 404 y sin rol admin 403; un nodo rechaza tokens de bloque ausentes o alterados y conexiones sin TLS; openssl confirma TLS 1.3.

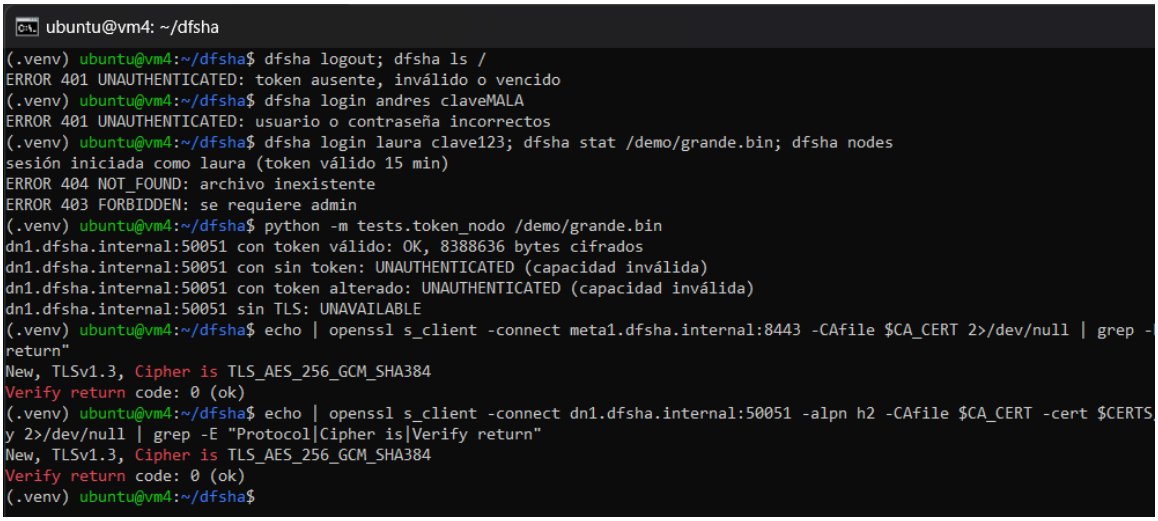


Figura 8. Accesos denegados, token alterado y TLS 1.3 en AWS.

9. Métricas en AWS

Corrida revision-bench-20261106T022920Z-5b3b1b: cinco repeticiones por condición desde el bastión, con muestras crudas, resumen, parámetros y hash del código. El throughput incluye cifrado, hashes, recibos firmados y publicación.

Exp.	Condición	Op.	MiB	Media (s)	p95 (s)	MiB/s
E1	R=3;bytes=10	put	10	0.54	0.58	18.6

Exp.	Condición	Op.	MiB	Media (s)	p95 (s)	MiB/s
E1	R=3;bytes=10	get	10	0.18	0.19	55.0
E1	R=3;bytes=100	put	100	2.90	3.23	34.8
E1	R=3;bytes=100	get	100	1.20	1.26	83.3
E1	R=3;bytes=500	put	500	16.61	18.76	30.2
E1	R=3;bytes=500	get	500	5.18	5.24	96.5
E2	R=2;W=2	put	100	2.69	2.75	37.2
E2	R=3;W=2	put	100	3.21	3.57	31.4
E3	clients=1	put	50	1.70	1.72	29.5
E3	clients=1	get	50	0.59	0.60	84.6
E3	clients=2	put	100	3.07	3.51	33.0
E3	clients=2	get	100	1.06	1.08	94.6
E3	clients=4	put	200	7.66	8.80	26.3
E3	clients=4	get	200	1.97	2.00	101.6
E3	clients=8	put	400	21.02	21.75	19.0
E3	clients=8	get	400	3.94	4.01	101.5

E1: con 500 MiB la escritura llega a 30.2 MiB/s y la lectura a 96.5 MiB/s; la lectura es más rápida porque no replica y descarga de réplicas distintas en paralelo. E2: R=2 sube a 37.2 MiB/s y R=3 a 31.4 MiB/s; la tercera copia cuesta poco porque la hace el nodo por S2S después de validar el bloque, no el cliente. E3: el throughput agregado de escritura pasa de 29.5 MiB/s con un cliente a 19.0 MiB/s con ocho, y el de lectura de 84.6 a 101.5 MiB/s. La lectura escala hasta saturar el bastión (2 vCPU, que también es nodo de datos); la escritura agregada baja con más clientes porque cada reserva, autorización y publicación compite por el epoch global del catálogo (R06): los conflictos se reintentan con espera aleatoria y se pierde tiempo en ellos. Es el costo medido de priorizar la corrección del catálogo; repartir el epoch por inodo es la mejora natural. La primera corrida en AWS falló justamente por ese motivo (409 RETRY sin reintento en el cliente) y se conserva en resultados. Con cinco muestras el p95 es el máximo observado.

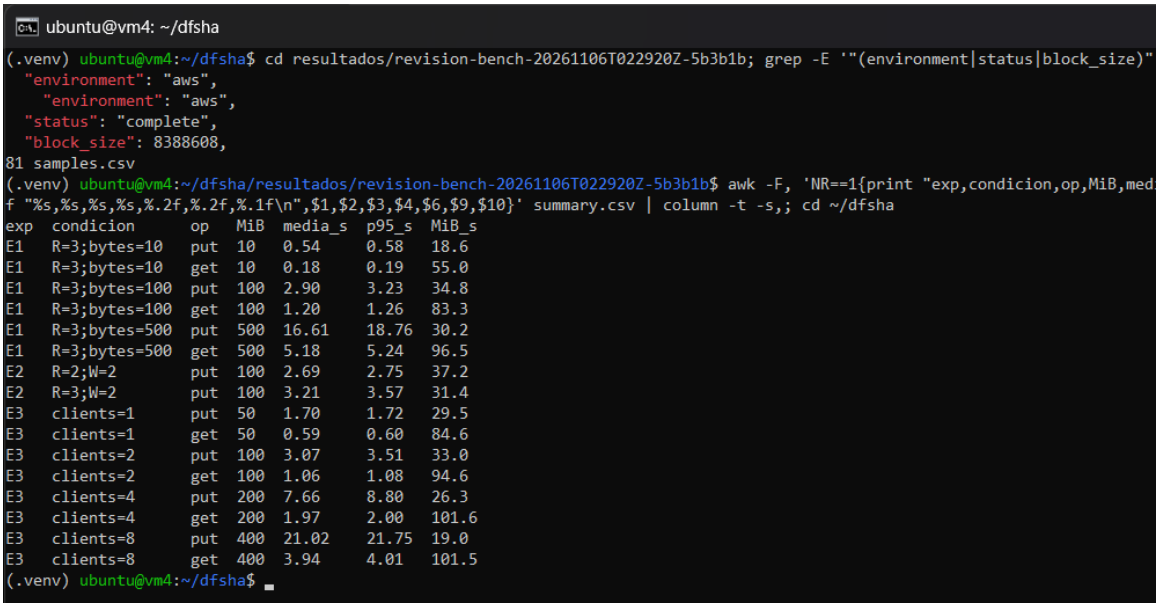


Figura 9. Resumen del benchmark en AWS consultado en el bastión.

10. Escenarios obligatorios de razonamiento

Escenario	Respuesta y comprobación
Mismo bloque simultáneo	Un lock excluye al segundo escritor (423). Si abrió sobre base anterior, se rechaza con 409 al adquirir. Se preserva contenido; se probó la base vieja y bloques distintos.
Réplica desconectada	La publicación exige dos recibos distintos, firmados y persistentes. Una sola copia no basta. El monitor repara con hash autorizado; los lectores usan réplicas de su snapshot.
Caída a mitad de bloque	El temporal no obtiene recibo ni se publica. La regresión envía un stream incompleto, comprueba DATA_LOSS y rechazo del commit. Reintento usa un plan nuevo.
Ubicación desconocida	El cliente usa nombres lógicos de coordinadores; open/manifest devuelve destinos y capacidades. El refresco conserva el mismo snapshot. extra_hosts se configura dentro de cada contenedor.

11. Entrega, contribuciones y defensa

Cada integrante registra sus aportes mediante commits en el repositorio del equipo. El guion del video de 10–15 minutos y la preparación de la defensa están en informe/GUION_VIDEO.md e informe/GUION_DEFENSA.md. Las simplificaciones declaradas en ADR_REVISION.md se someten a aprobación del profesor.

Referencias y fuentes de comparación

Los documentos individuales de la Fase I son las fuentes de la comparación.

[1] S. Ghemawat, H. Gobioff y S.-T. Leung, "The Google file system," en Proc. 19th ACM Symp. Operating Systems Principles, 2003, pp. 29-43.

[2] K. Shvachko, H. Kuang, S. Radia y R. Chansler, "The Hadoop distributed file system," en Proc. IEEE MSST, 2010, pp. 1-10.

[3] etcd Authors, "etcd documentation," 2026. Disponible: https://etcd.io/docs/
[4] A. Ospina Sanjuan, "gRPC: de la invocación local a los microservicios modernos," diapositivas de clase, Universidad Pontificia Bolivariana, 2026.
[5] J. H. Howard et al., "Scale and performance in a distributed file system," ACM Trans. Computer Systems, vol. 6, no. 1, pp. 51-81, 1988.
[6] M. Jones, J. Bradley y N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, 2015.
[7] E. Rescorla, "The Transport Layer Security (TLS) protocol version 1.3," RFC 8446, IETF, 2018.
[8] M. Dworkin, "Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC," NIST SP 800-38D, 2007.
[9] D. Ongaro y J. Ousterhout, "In search of an understandable consensus algorithm," en Proc. USENIX ATC, 2014, pp. 305-319.
[10] E. Brewer, "CAP twelve years later: How the 'rules' have changed," Computer, vol. 45, no. 2, pp. 23-29, 2012.
[11] M. van Steen y A. S. Tanenbaum, Distributed Systems, 4.ª ed. distributed-systems.net, 2023.